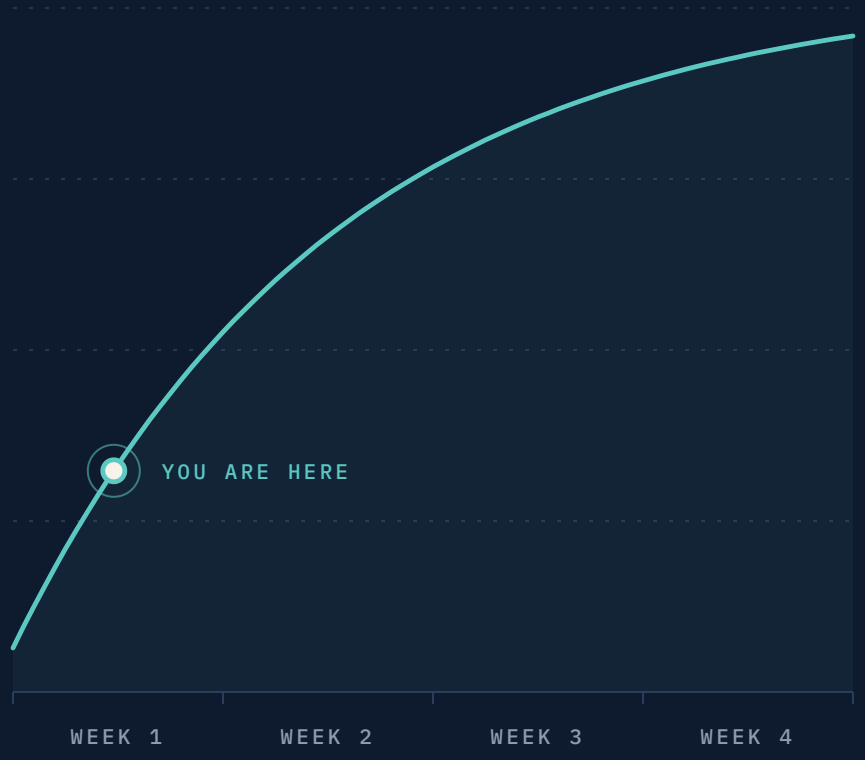
WEEK 1 · DAY 2 · OVERVIEW

Files and Folders



Created by **Jeff Lenamon**

ABOUT 55 MIN WITH EXERCISES / 8 SECTIONS / 7 EXERCISES

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

The Desk Lead's Fourth Note

The book lives only while your notebook runs: restart it and the four bonds are gone. Give it a folder of its own, write it to a file there, and show me it reads back to 3,000,000 face and 2,958,750 dollars of cost.

Your Route Through Day 2

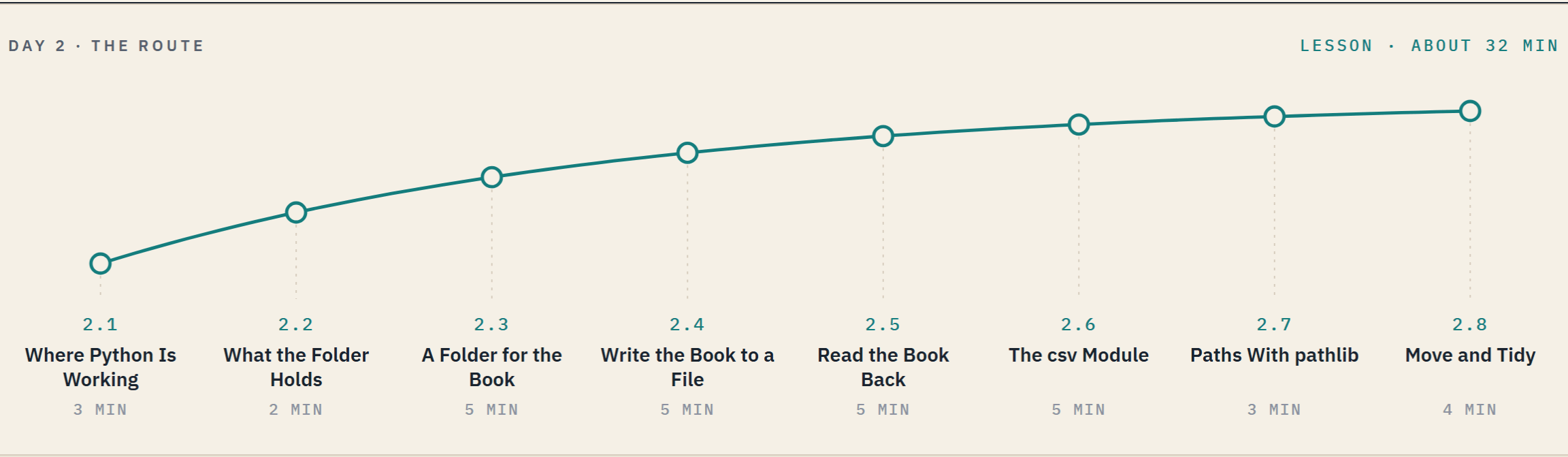


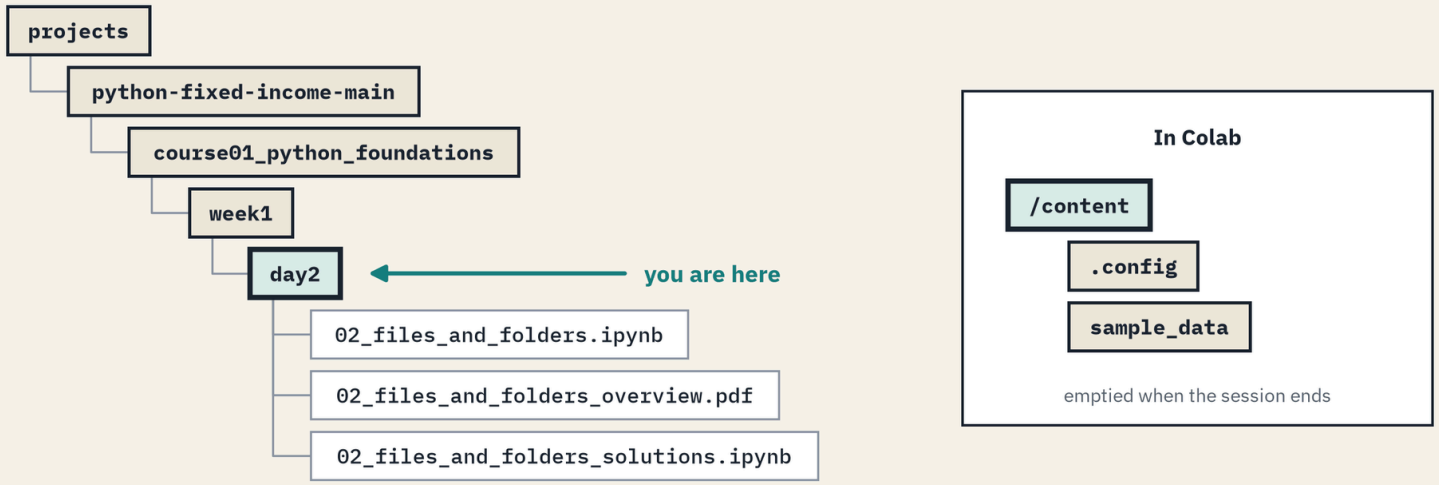
FIGURE 1 Eight sections, in order, with about how long each takes.

Time. About 32 minutes for the lesson, then about 21 for the seven exercises.

2.1 Where Python Is Working

The working directory: where Python reads and writes

`C:\Users\you\projects\python-fixed-income-main\course01_python_foundations\week1\day2`



a file name with no folder in front is looked for in day2

FIGURE 2 The folders from projects down to day2, marked you are here; in Colab the working directory is /content.

ASK THE SYSTEM

```
import os
print(os.name)
```

```
nt
```

EXCEL SIDE BY SIDE

Excel side by side. In desktop Excel, `=CELL("filename",A1)` shows a saved workbook's full path, file and sheet name at the end.

On your own machine VS Code starts Python in the notebook's own folder, `day2`**.**

2.2 What the Folder Holds

LIST THE FOLDER

```
names = os.listdir('.')
print(len(names))
```

```
3
```

WALK THE NAMES, KEEP THE NOTEBOOKS

```
for name in names:
    print(name)
notebooks = [name for name in names if name.endswith('.ipynb')]
print(notebooks)
```

```
02_files_and_folders.ipynb
02_files_and_folders_overview.pdf
02_files_and_folders_solutions.ipynb
['02_files_and_folders.ipynb', '02_files_and_folders_solutions.ipynb']
```

`listdir()` **returns a list of names, in no set order: here the notebook, its solutions, and its overview PDF.**

2.3 A Folder for the Book

MAKE THE FOLDER

```
os.mkdir('desk_holdings')
print(os.path.exists('desk_holdings'))
```

```
True
```

CHECK FIRST

```
if not os.path.exists('desk_holdings'):
    os.mkdir('desk_holdings')
    print('Folder created')
else:
    print('Folder already exists')
```

```
Folder already exists
```

RUN IT TWICE

```
os.mkdir('desk_holdings')
```

```
FileExistsError: [WinError 183] Cannot create a file when that file already exists: 'desk_holdings'
```

Checking first makes the cell safe to rerun.

A Path Built, Not Typed

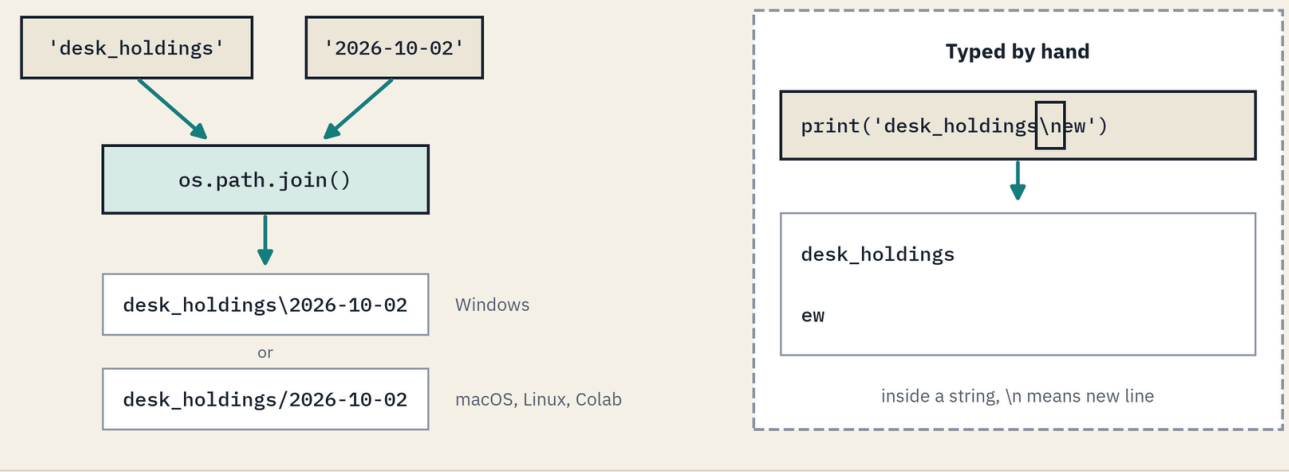


FIGURE 3 os.path.join() puts in the right slash for each machine; typed by hand, \n splits the path in two.

BUILD THE PATH

```
day_folder = os.path.join('desk_holdings', '2026-10-02')
print(day_folder)
os.mkdir(day_folder)
print(os.listdir('desk_holdings'))
```

```
desk_holdings\2026-10-02
['2026-10-02']
```

TYPE IT BY HAND

```
print('desk_holdings\new')
```

```
desk_holdings
ew
```

Hand-typed Windows paths are a classic bug; `os.path.join()` never has it.

2.4 Write the Book to a File

holdings.csv, as read() gives it

```
Ticker,Coupon,Price,Face \n
QVNE,5.25,98.5,1500000 \n
DNMR,6.125,103.25,500000 \n
PLWK,4.75,95.0,750000 \n
HLVF,7.5,101.0,250000 \n
```

the same file in Excel

	A	B	C	D
1	**Ticker**	**Coupon**	**Price**	**Face**
2	QVNE	5.25	98.5	1500000
3	DNMR	6.125	103.25	500000
4	PLWK	4.75	95	750000
5	HLVF	7.5	101	250000

FIGURE 6 holdings.csv as five lines of text with commas, and the same file as a spreadsheet grid.

OPEN, WRITE, CLOSE

```
file_path = os.path.join(day_folder, 'holdings.csv')
print(file_path)
file = open(file_path, 'w', encoding='utf-8')
file.write('Ticker,Coupon,Price,Face\n')
file.close()
```

```
desk_holdings\2026-10-02\holdings.csv
```

EXCEL SIDE BY SIDE

Excel side by side. Double-click **`holdings.csv`** and Excel shows five rows and four columns, splitting each line at the commas.

`'w'` creates the file, or wipes it if it is there.

Let a Loop Write the Bonds

A LOOP WRITES THE BONDS

```
with open(file_path, 'a', encoding='utf-8') as file:
    for ticker, coupon, price, face in zip(ticker_list, coupon_list, price_list, face_list):
        file.write(f'{ticker},{coupon},{price},{face}\n')
```

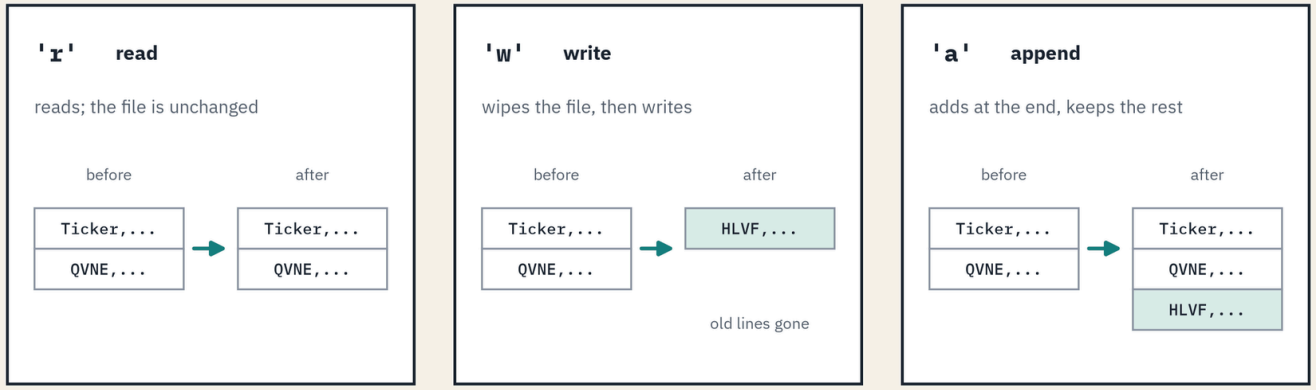


FIGURE 5 Three modes: r reads and leaves the file alone, w wipes it then writes, a adds at the end.

READ IT BACK

```
with open(file_path, 'r', encoding='utf-8') as file:
    print(file.read())
```

```
Ticker,Coupon,Price,Face
QVNE,5.25,98.5,1500000
DNMR,6.125,103.25,500000
PLWK,4.75,95.0,750000
HLVF,7.5,101.0,250000
```

Four bonds from one loop: Part 2's promise kept.

2.5 Read the Book Back

TOTAL THE FACE

```
total_face = 0
for line in lines[1:]:
    values = line.strip().split(',')
    total_face += int(values[3])
print(f'The total face held is {total_face:,}')
```

```
The total face held is 3,000,000
```

EXCEL SIDE BY SIDE

Excel side by side. `split(',')` is **Text to Columns** on Excel's Data tab, with the comma as the delimiter.

ONE LINE, STEP BY STEP

STEP	GIVES
`line`	`'QVNE,5.25,98.5,1500000\n'`, one string
`line.strip()`	`'QVNE,5.25,98.5,1500000'`, the `\n` gone
`.split(',')`	`['QVNE', '5.25', '98.5', '1500000']`, four strings
`values[3]`	`'1500000'`, still text: the `TypeError`
`int(values[3])`	`1500000`, a number to add

A `TypeError`: Python will not add the text '1500000' to the number 0.

2.6 The csv Module

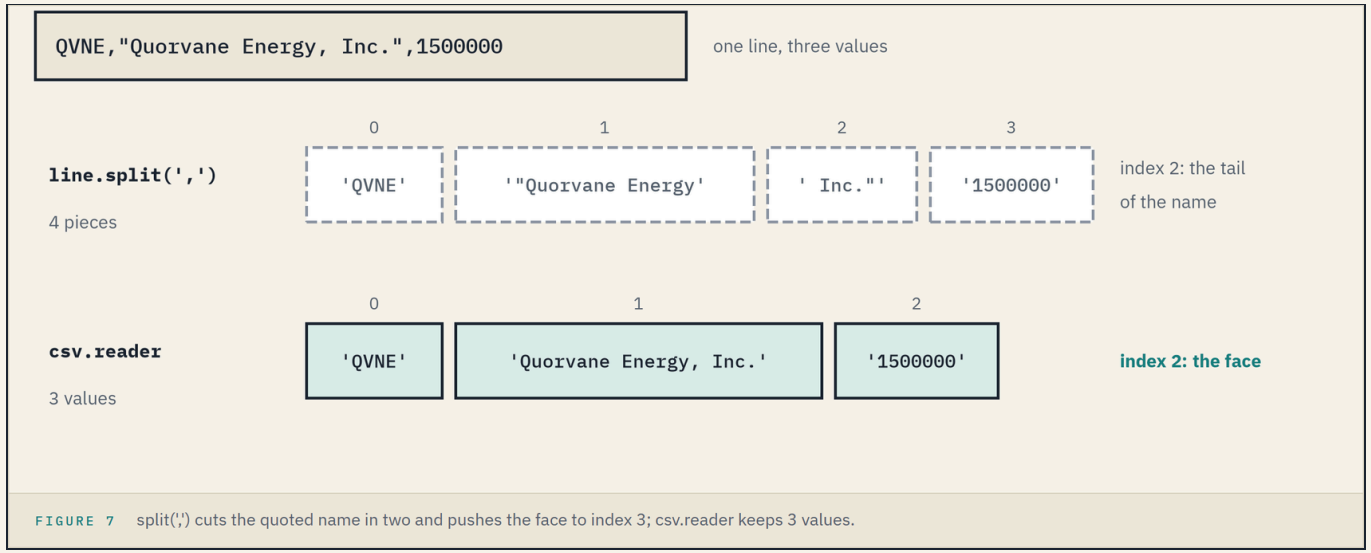


FIGURE 7 split(',') cuts the quoted name in two and pushes the face to index 3; csv.reader keeps 3 values.

SPLIT AT THE COMMAS

```
values = line.split(',')
print(values)
```

```
['QVNE', '"Quorvane Energy', ' Inc."', '1500000']
4 pieces; index 2 holds  Inc."
```

CSV.READER

```
for values in csv.reader([line]):
    print(values)
```

```
['QVNE', 'Quorvane Energy, Inc.', '1500000']
3 values; index 2 holds 1500000
```

`csv.reader`: 3 values, the quoted comma kept in the name, the face back at index 2.

Read by Column Name

TOTAL THE COST BY COLUMN NAME

```
total_cost = 0
with open(file_path, 'r', encoding='utf-8', newline='') as file:
    for row in csv.DictReader(file):
        total_cost += int(row['Face']) * float(row['Price']) / 100
print(f'The book cost ${total_cost:,.2f}')
```

```
The book cost $2,958,750.00
```

THE WHOLE FILE IN ONE LINE

```
import pandas as pd
pd.read_csv(file_path)
```

	Ticker	Coupon	Price	Face
0	QVNE	5.250	98.50	1500000
1	DNMR	6.125	103.25	500000
2	PLWK	4.750	95.00	750000
3	HLVF	7.500	101.00	250000

pandas reads the whole file into a table in one line, the numbers already numbers: week 2 starts there.

2.7 Paths With pathlib

ASK THE PATH

```
print(holdings_path.exists())
print(holdings_path.name)
print(holdings_path.parent)
print(holdings_path.parent.name)
```

```
True
holdings.csv
desk_holdings\2026-10-02
2026-10-02
```

OS AGAINST PATHLIB

JOB	WITH OS	WITH PATHLIB
Join the parts of a path	`os.path.join('desk_holdings', '2026-10-02')`	`Path('desk_holdings') / '2026-10-02'`
Check that it exists	`os.path.exists(path)`	`path.exists()`
Take the last part	`os.path.basename(path)`	`path.name`
Find the files that match	`os.listdir()` with `endswith()`	`path.glob('*.csv')`

A `Path` works in `open()`, and in pandas next week, wherever a text path does.

2.8 Move and Tidy

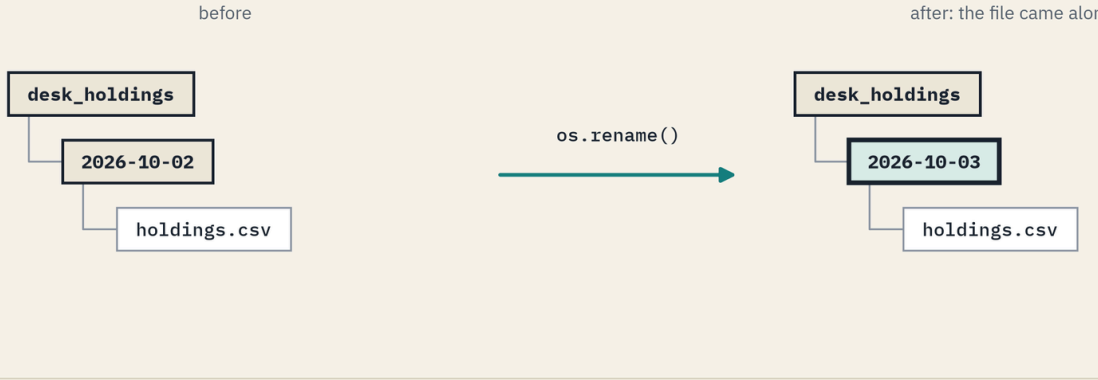


FIGURE 8 os.rename() turns 2026-10-02 into 2026-10-03 and holdings.csv moves with it.

RENAME

```
os.rename(day_folder, new_day_folder)
print(os.listdir('desk_holdings'))
print(os.listdir(new_day_folder))
```

```
['2026-10-03']
['holdings.csv']
```

REMOVE A FOLDER THAT IS NOT EMPTY

```
os.rmdir('desk_holdings')
```

```
OSError: [WinError 145] The directory is not empty: 'desk_holdings'
```

INNER FOLDER FIRST

```
os.rmdir(new_day_folder)
os.rmdir('desk_holdings')
print(os.path.exists('desk_holdings'))
```

```
False
```

`rmdir()` refuses a folder that is not empty, a safety feature: the inner folder had to go first.

You Can Now

- See which folder Python works in and what it holds.
- Make, rename, and remove folders, with paths that work on any machine.
- Write a file, its rows by a loop.
- Read a file line by line or with **`csv`**, and total a column.
- Turn text from a file into numbers before you calculate.

THE NOTE, ANSWERED

THE NOTE ASKED	ANSWER
A folder of its own	**`desk_holdings`**, a folder per day inside
Write it to a file	**`holdings.csv`**, four bonds by one loop
3,000,000 face read back	3,000,000, by **`split`**
2,958,750 dollars of cost read back	2,958,750.00, by **`csv.DictReader`**